USER & TECHNICAL HANDBOOK

testsweet-Handbuch

Mitarbeiter-Simulator & Integrationstest für SDN.Leistung — die Anwendung verstehen und technisch durchdringen

> Projekt Leistungsmanagement Grundsicherung (SGB XII)
>
> Teil I: Die Anwendung — Bedienung, Kategorien, Ergebnisse
>
> Teil II: Technische Architektur — Stack, Klassen, Abläufe
>
> Stand: 12. Oktober 2026 (Version 0.5.8)

Inhaltsverzeichnis

1 testsweet in Kürze 3

2 Voraussetzungen und Start 3

2.1 Was einmalig vorbereitet sein muss 3
2.2 Die Oberfläche starten (empfohlen) 4
2.3 Ohne Oberfläche: Batch-Läufe 4

3 Die Oberfläche im Überblick 5

4 Einen Testlauf fahren 6

5 Die Ergebnisse lesen 8

5.1 Kenndaten des Laufs 8
5.2 Auftragsliste und Live-Protokoll 9
5.3 Das Log-Verzeichnis — der eigentliche Zweck 10

6 Die vier Testfallkategorien 11

6.1 Positivfälle — die Erfolgswege 11
6.2 Negativfälle — die Abwehrseiten 11
6.3 Lasttest — viele Fälle, kurzer Bericht 12
6.4 Stresstest — zwei Wellen über dieselben User 12

7 Regeln für den sicheren Umgang 12

8 Architekturüberblick 14

9 Der Technologiestack 15

10 Die Klassen 16

10.1 Die wichtigsten Klassen im Detail 17

11 Abläufe im Detail 18

12 Die eigenen Tests des Simulators und bekannte Grenzen 21

Hinweis: Dieses Inhaltsverzeichnis wird über Feldfunktionen erzeugt. Nach Änderungen am Dokument bitte mit Rechtsklick auf das Verzeichnis und Felder aktualisieren wählen, damit die Seitenzahlen stimmen.

1 testsweet in Kürze

testsweet ist ein Mitarbeiter-Simulator und Integrationstest für die Fachanwendung SDN.Leistung (Leistungsmanagement Grundsicherung, SGB XII). Der Name ist Programm: Der Simulator stellt *N simulierte Sachbearbeiter* ein, die parallel einen gemeinsamen Topf von Anwenderszenarien abarbeiten — Anlage von Vorgängen, Auszahlungen anordnen, Vier-Augen-Freigaben, Storni — und danach wird auf der Oracle-Datenbank geprüft, ob auch wirklich das Richtige angekommen ist.

Wichtig zur Einordnung: Das ist **kein Unit-Test** und nicht Teil von `mvn test` im Hauptprojekt. testsweet ist ein eigenständiges Werkzeug, das den ganzen Stack von der Fachschicht bis zur Datenbank (oder im Modus 2 sogar bis zur echten Weboberfläche im Browser) in Bewegung setzt. Die Auswertung erfolgt bewusst manuell und gemeinsam — der Simulator liefert das Rohmaterial: Kenndaten, Protokolle, Screenshots, eine PDF-Testdokumentation und eine maschinenlesbare Markdown-Fassung.

Der Simulator kennt **zwei Betriebsarten**: Im **Modus 1** (Fachschicht) rufen die simulierten Sachbearbeiter den Produktionscode direkt auf (`SozialLeistungsService` → `TxManager` → Oracle) und prüfen damit Sperren, Deadlock-Wiederholung, Budgetführung, Vier-Augen-Prinzip und Lost-Update-Schutz. Im **Modus 2** (Weboberfläche) steuert jeder simulierte Sachbearbeiter ein eigenes sichtbares Chrome-Fenster über Selenium und bedient die echte SDN.Leistung-Oberfläche wie ein Mensch — inklusive Screenshots je Schritt.

Und er kennt **vier Testfallkategorien**: Positivfälle (Erfolgswege mit vollständiger Prüfung), Negativfälle (erwartete Fachfehler mit unverändertem Datenbank-Endzustand), Lasttest (schlanke Massendurchläufe mit Aggregate-Report) und Stresstest (zwei Wellen über dieselben Bestände mit Phasenbarriere). Alles läuft lokal: Die Weboberfläche des Simulators lauscht ausschließlich auf 127.0.0.1, Port 8095.

2 Voraussetzungen und Start

2.1 Was einmalig vorbereitet sein muss

- `JDK 21` und `Maven` — dieselbe Umgebung wie für das Hauptprojekt.
- **Oracle-Datenbank**: die Entwickler-Datenbank im Docker-Container `lmg-oracle` (Start über `start.bat` im Hauptprojekt). Der Zugang läuft über `TX_URL`, `TX_USER`, `TX_PASS`; Standard ist `jdbc:oracle:thin:@//localhost:1521/FREEPDB1` mit tx_demo/tx_demo.
- **Nach Änderungen am Hauptprojekt**: einmal `mvn install` im Hauptverzeichnis ausführen, damit der Simulator die aktuelle Version der Fachschicht (`transaktionssteuerung`) benutzt.

- **Nur für Modus 2**: ein installiertes Google Chrome; der passende ChromeDriver wird über WebDriverManager automatisch beschafft.

2.2 Die Oberfläche starten (empfohlen)

Unter Windows genügt `testsweet\start.bat` — das Fenster bleibt als Konsole offen und der Browser öffnet die Oberfläche automatisch unter `http://127.0.0.1:8095/`. Alternativ plattformunabhängig: `mvn -f testsweet/pom.xml compile exec:java`. Ein anderer Port lässt sich über die Umgebungsvariable `SWEET_PORT` setzen.

2.3 Ohne Oberfläche: Batch-Läufe

Für automatisierte Abnahmen gibt es zwei Kommandozeilen-Varianten. **Fachschicht (Modus 1)**: `testsweet\start.bat run <mitarbeiter> <auftraege> [seed] [denkzeitMs] [kategorie]` — zum Beispiel `start.bat run 10 60 42 0` für zehn parallele Mitarbeiter, 60 Aufträge, Seed 42, Positivfälle. **Weboberfläche (Modus 2)**: `start.bat web <mitarbeiter> <auftraege> [seed] [denkzeitMs] [kategorie]` — startet bei Bedarf automatisch den SDN.Leistung-Server auf Port 8080, ordnet die Chrome-Fenster nebeneinander an und spielt die Vorgänge durch. Batch-Läufe beenden sich mit **Exitcode 1**, wenn der Lauf oder eine globale Prüfung fehlschlägt — so kann man sie in Skripte und CI einhängen.

3 Die Oberfläche im Überblick

Die Oberfläche ist ein einziges langes Dashboard mit fünf Sprungmarken in der Reiterleiste: **Konfiguration**, **Kenndaten**, **Architektur & Diagramme**, **Auftragsliste** und **Live-Protokoll**. Ganz oben stehen Titel, der Status-Indikator des Laufs (LÄUFT, FERTIG, FEHLGESCHLAGEN) und die Kurzform der Datenbank-Verbindung.

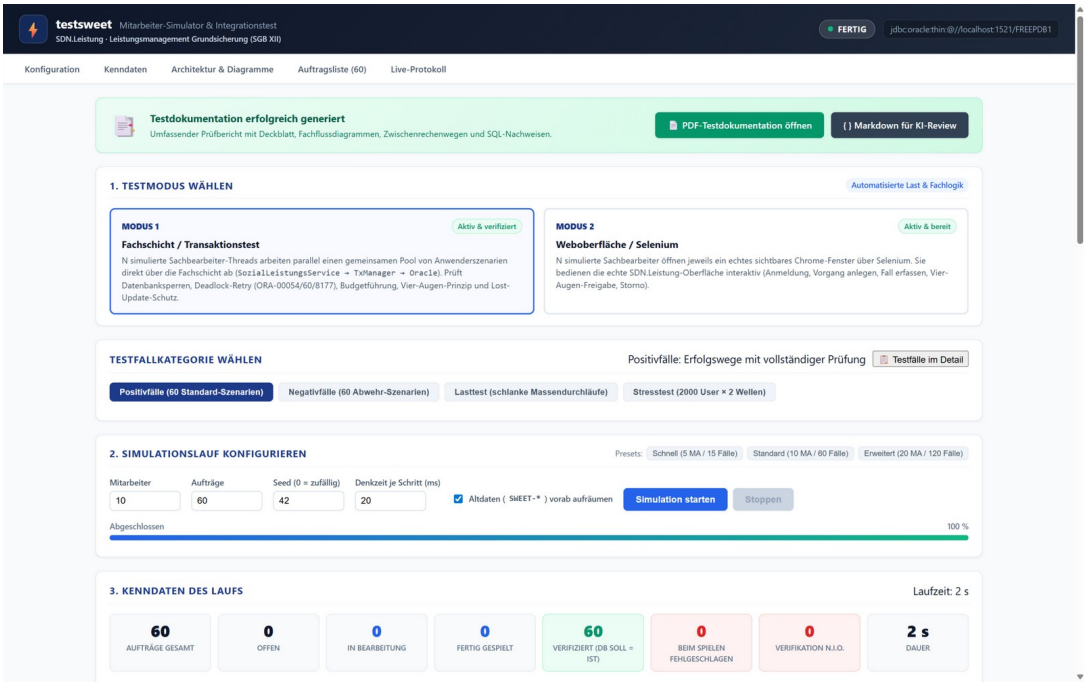


Abbildung 1: Konfigurationsbereich — Statuszeile, Testdokumentation-Banner, Moduswahl, Testfallkategorie und Laufkonfiguration

Der auffälligste Bereich ist das grüne Testdokumentation-Banner über der Konfiguration: Nach jedem Lauf bietet es die erzeugte Prüfdocumentation direkt an — links der Knopf *PDF-Testdokumentation öffnen* (Deckblatt, Fachflussdiagramme, Zwischenrechenwege und SQL-Nachweise je Testfall), daneben *Markdown für KI-Review* (dieselben Daten maschinenlesbar, inklusive der vergebenen Datenbank-IDs und Prüf-SQLs).

Darunter liegen die drei Konfigurations-Abschnitte: **1. Testmodus wählen** (zwei Karten: Modus 1 Fachschicht, Modus 2 Weboberfläche/Selenium), **Testfallkategorie wählen** (vier Knöpfe, siehe Kapitel 6) und **2. Simulationslauf konfigurieren** (Mitarbeiter, Aufträge, Seed, Denkzeit, Aufräum-Checkbox plus Schnell-Presets). Die Abschnitte **3. Kenndaten des Laufs** bis **6. Live-Transaktionsprotokoll** füllen sich ab dem ersten Lauf.

Ein Hinweis zur Erwartung: Die Kenndaten zeigen immer nur den letzten Lauf dieser GUI-Sitzung (live aus dem Speicher) — keine Tagesübersicht über alle Läufe. Die vollständige Geschichte liegt im Log-Verzeichnis jedes Laufs.

4 Einen Testlauf fahren

Ein Lauf ist bewusst ein Dreiklick-Erlebnis — Modus wählen, Kategorie wählen, Starten:

1. **Modus wählen:** Karte **Modus 1 (Fachschicht)** für Transaktionstests direkt am Produktionscode, oder **Modus 2 (Weboberfläche/Selenium)** für den Durchlauf durch die echte Benutzeroberfläche mit sichtbaren Chrome-Fenstern. Die gewählte Karte ist blau umrandet.
2. **Kategorie wählen:** Einer der vier Knöpfe — **Positivfälle**, **Negativfälle**, **Lasttest** oder **Stresstest**. Das Beschriftungsfeld neben der Überschrift erklärt die gewählte Kategorie; der Knopf *Testfälle im Detail* öffnet einen Dialog mit der vollständigen, verständlich geschriebenen Beschreibung jedes Szenario-Typs samt erwartetem Datenbank-Endzustand.
3. **Konfigurieren: Mitarbeiter** = Anzahl paralleler simulierter Sachbearbeiter. **Aufträge** = Größe des Auftragstopfs (beim Stresstest: die Anzahl der User, je User laufen zwei Tests). **Seed** — 0 bedeutet zufällig, ein fester Seed erzeugt dieselben Szenarien wieder und macht einen Lauf reproduzierbar. **Denkzeit je Schritt** (ms) bremst die simulierten Mitarbeiter ab, sinnvoll vor allem im Modus 2. Die Checkbox *Altdaten (SWEET-*) vorab aufräumen* (Standard: an) entfernt Reste früherer Läufe vor dem Start. Die Presets rechts setzen bewährte Kombinationen — die Auswahl passt sich der Kategorie an.
4. **Starten und beobachten:** *Simulation starten* beginnt den Lauf; der Fortschrittsbalken und das Live-Protokoll zeigen den Fortgang, *Stoppen* bricht kontrolliert ab. Im Modus 2 öffnen sich zusätzlich die Chrome-Fenster der simulierten Sachbearbeiter im Bildschirmraster.
5. **Ergebnis entgegennehmen:** Am Ende springt das Testdokumentation-Banner an (PDF/Markdown), die Kenndaten zeigen den frischen Lauf, und im Ordner `testsweet/logs/run-<Zeitstempel>/` liegt das vollständige Material.

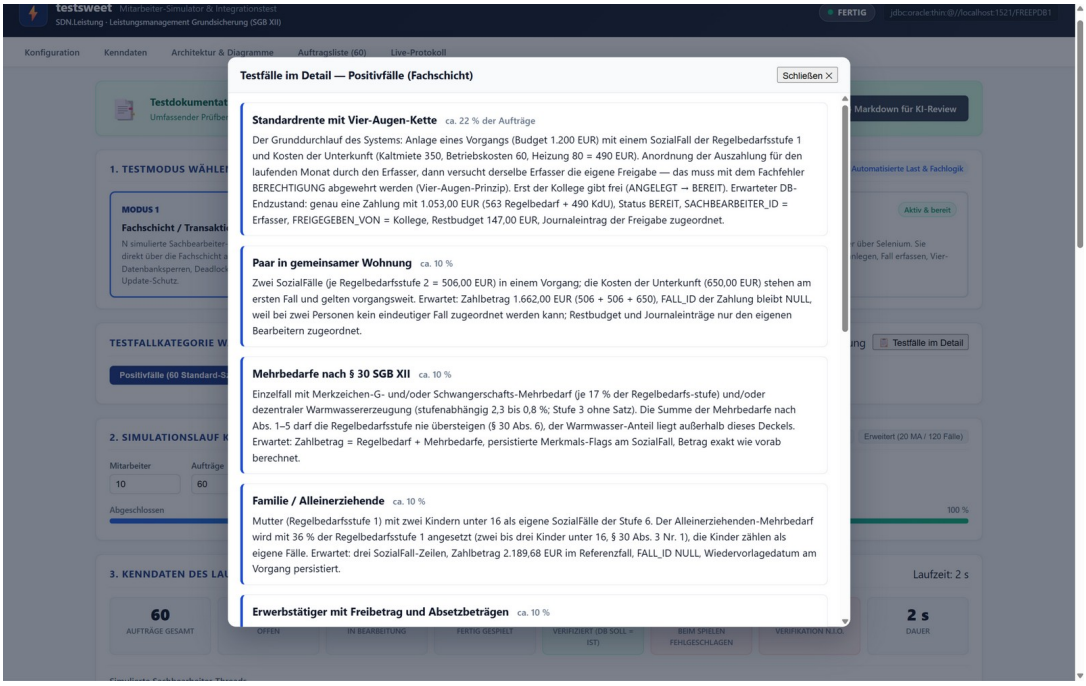


Abbildung 2: Der Dialog „Testfälle im Detail“ — jeder Szenario-Typ mit Erklärung und erwartetem Datenbank-Endzustand

5 Die Ergebnisse lesen

5.1 Kenndaten des Laufs

Die Kenndaten-Karten fassen den Lauf in acht Zahlen zusammen: Aufträge gesamt, offen, in Bearbeitung, fertig gespielt, **verifiziert (DB Soll = Ist)**, beim Spielen fehlgeschlagen, Verifikation nicht in Ordnung, und die Dauer. Zwei Karten werden gern verwechselt — sie prüfen aber völlig Verschiedenes: **Beim Spielen fehlgeschlagen** heißt, der Durchlauf des simulierten Sacharbeiters ist abgebrochen (etwa ein eingefrorenes Browser-Fenster im Modus 2); die Datenbank kann trotzdem korrekt sein. **Verifikation nicht in Ordnung** heißt, der Datenbank-Endzustand weicht von der Erwartung ab — das ist die ernste Meldung.

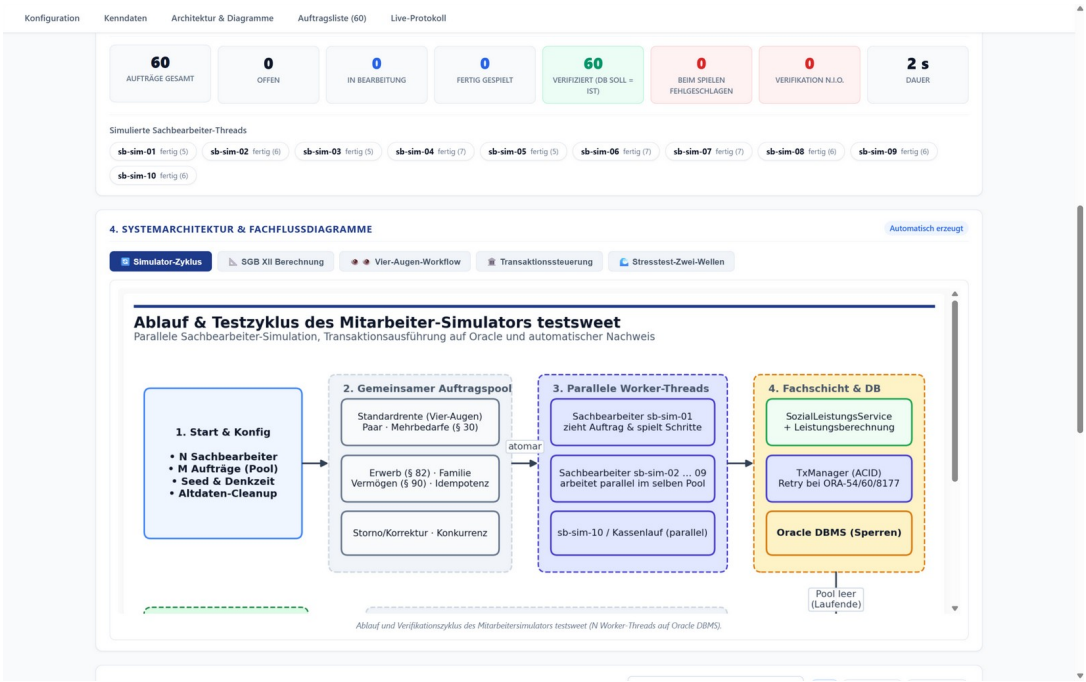


Abbildung 3: Kenndaten-Karten und simulierte Sachbearbeiter-Threads, dahinter die Diagramm-Galerie

Darunter zeigt jede Zeile einen **simulierten Sachbearbeiter-Thread** (sb-sim-01, sb-sim-02, …) mit seinem Zustand und der Anzahl abgeschlossener Aufträge — so erkennt man schnell, ob die Arbeit gleichmäßig verteilt war oder ein Thread hing.

5.2 Auftragsliste und Live-Protokoll

Die **Auftragsliste** führt jeden Auftrag mit Nummer, Szenario-Typ, Aktenzeichen (SWEET-…), Hauptperson, Bearbeiter, Status und Dauer; das Suchfeld filtert nach Aktenzeichen, Name oder Szenario, die Knöpfe *Alle*, *Nur Verifiziert* und *Auffälligkeiten* schneiden die Menge zu. Ein Doppelklick auf eine Zeile öffnet die vollständige Schritt- und Rechenchronologie des Auftrags — inklusive der Zwischenwerte der Bedarfsberechnung.

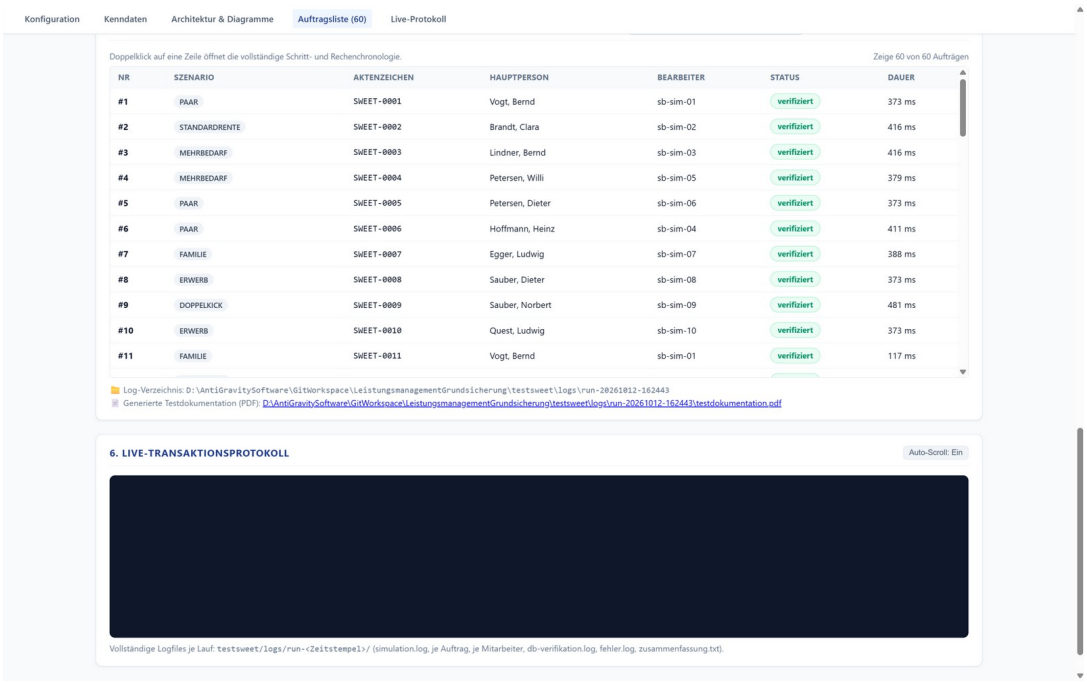


Abbildung 4: Auftragsliste mit Suche, Filtern und dem Sprung zum Log-Verzeichnis des Laufs

Das **Live-Transaktionsprotokoll** streamt während des Laufs jedes Ereignis in ein dunkles Konsolenfeld (mit umschaltbarem Auto-Scroll). Nach dem Lauf ist die Box leer — alles steht dann in den Logdateien des Lauf-Verzeichnisses.

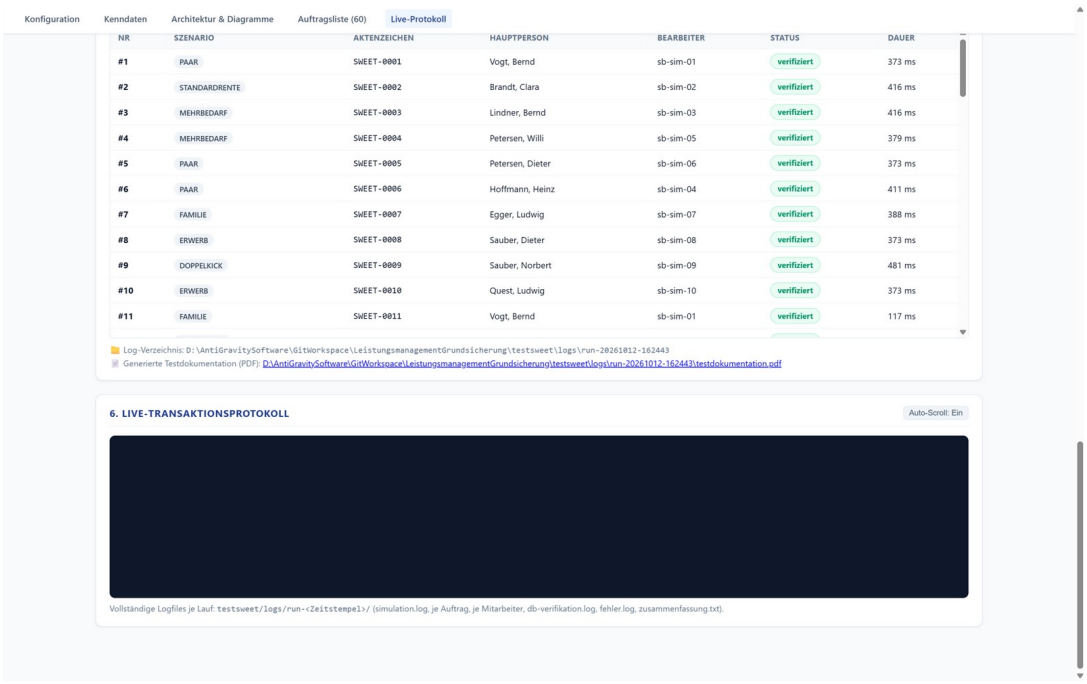


Abbildung 5: Live-Protokoll — während des Laufs streamt hier das Ereignislog; danach lebt es nur noch in den Logdateien

5.3 Das Log-Verzeichnis — der eigentliche Zweck

Je Lauf entsteht ein Ordner `testsweet/logs/run-<Zeitstempel>/` mit dem vollständigen Material für die gemeinsame Auswertung:

Tabelle 1: Dateien im Lauf-Verzeichnis

Datei	Inhalt
simulation.log	alle Ereignisse chronologisch (Millisekunden, Kanal, Mitarbeiter)
mitarbeiter-<kürzel>.log	alles eines simulierten Sachbearbeiters
auftrag-<id>-<aktenzeichen>.log	Schrittprotokoll eines einzelnen Auftrags
db-verifikation.log	Erwartung vs. Ist je Prüfung (OK/Abweichung)
fehler.log	nur Fehler und Abweichungen
zusammenfassung.txt	Kenndaten des Laufs (Modus, Mitarbeiter, Verifikationsquote)
screenshots/	nur Modus 2: Screenshot je UI-Schritt (tf-NN-schritt-MM.png)
testdokumentation.pdf / .md	Positiv/Negativ: vollständiger Prüfbericht je Fall (PDF für Menschen, Markdown zur KI-Nachprüfung)
lastreport.pdf / .md	Last/Stress: gekürzter Gesamtreport mit Aggregaten statt Einzelfällen

6 Die vier Testfallkategorien

Die Kategorie bestimmt, welche Szenarien in den Auftragstopf wandern, welches Aktenzeichen-Präfix die Daten erhalten (dadurch lassen sich die Mengen in der Datenbank auseinanderhalten) und wie der Bericht aussieht:

Tabelle 2: Kategorien im Überblick

Kategorie	Modus 1 (Fachschicht)	Modus 2 (Web)	Bericht
Positivfälle	SWEET- · 10 Szenario-Arten	SWEET-W- · Web-Varianten	testdokumentation.pdf/.md je Fall
Negativfälle	SWEET-N- · 9 Abwehr-Arten	SWEET-NW- · 7 Web-Arten	testdokumentation.pdf/.md je Fall
Lasttest	SWEET-L- · Massendurchläufe	SWEET-LW-	lastreport.pdf/.md (nur Aggregate)
Stresstest	SWEET-S- · 2 Wellen	— (bewusst nur Fachschicht)	lastreport.pdf/.md (nur Aggregate)

6.1 Positivfälle — die Erfolgswege

Zehn Szenario-Arten decken das fachliche Herz des Systems ab: Standardrente (die Vier-Augen-Kette), Paar in gemeinsamer Wohnung, Mehrbedarf nach § 30, Familie/Alleinerziehende, Erwerbstätiger mit Freibeträgen (§ 82), Vermögen mit Schonvermögen (§ 90, inklusive einer Variante mit erwartetem KEIN_ANSPRUCH), Doppelklick (Idempotenz), Storno-Korrektur (Storno und Neuauszahlung im selben Monat), Konkurrenz (erwarteter Lost-Update-Schutz zweier gleichzeitiger Bearbeiter) und Kassenlauf (läuft parallel zu Freigaben). Im Modus 2 kommen Web-Varianten hinzu (WEB-STANDARDRENTE, WEB-MEHRBEDARF, WEB-STORNO). Das Besondere an der Prüfung: Die **Erwartungshaltung** — Beträge, Status, Restbudget, Versionen — wird vor dem Lauf mit derselben reinen Berechnung (`Leistungsberechnung`) ermittelt, die auch die Fachschicht nutzt. Der Simulator rechnet also nicht mit einer zweiten, abweichenden Logik.

6.2 Negativfälle — die Abwehrseiten

Neun Fachschicht-Arten prüfen, dass das System Unzulässiges zuverlässig ablehnt: Rente über Bedarf (§ 43 Abs. 1), Startbudget unter dem Zahlbetrag, Doppelanordnung je Monat, Selbstfreigabe (Vier-Augen-Verletzung), Freigabe einer bereits BEREIT-Zahlung, Storno nach Kassenübergabe, ungültige IBAN-Prüfziffer, umgekehrter oder überlanger Bescheid-Zeitraum (§ 44 Abs. 3) und Operationen auf nicht existierenden IDs. Der erwartete Fachfehler ist Teil des Szenarios — und zusätzlich wird geprüft, dass die Datenbank *unverändert* bleibt: keine Zahlung, kein Bescheid, Budget unangetastet. Die sieben Web-Arten machen dasselbe auf der Oberfläche (falsches Passwort, ungültige Bankverbindung, Anordnung ohne Zahlungsmonat, …) und bewiesen unter anderem, dass der Storno-Knopf einer freigegebenen Zahlung der Sachbearbeiter-Rolle gar nicht angeboten wird.

6.3 Lasttest — viele Fälle, kurzer Bericht

Der Lasttest fährt schlanke Durchläufe (Vorgang + SozialFall + Anordnung, ohne Freigabe und Storno) in hoher Stückzahl. Weil bei 600+ Fällen eine Einzelfall-Dokumentation ausufern würde, schreibt der Simulator bewusst nur einen **Gesamtreport** mit Aggregaten: Kennzahlen, Durchsatz (Aufträge/min), Dauer je Auftrag (min/Ø/max), die Summe der verifizierten Auszahlungen und die Verteilung je Szenario-Typ und Sachbearbeiter. Die Einzelnachweise stehen vollständig in den Protokolldateien des Lauf-Verzeichnisses. Referenzlauf: 600 Fälle mit 20 Mitarbeitern in rund 3 Sekunden gegen die Docker-Oracle.

6.4 Stresstest — zwei Wellen über dieselben User

Der Stresstest läuft bewusst nur im Fachschicht-Modus (GUI und Schnittstelle wehren Modus 2 ab) und arbeitet in **zwei Wellen über dieselben Bestände**: Welle 1 legt je User Vorgang und SozialFall an — noch ohne Zahlung. Erst wenn Welle 1 vollständig abgeschlossen ist (Phasenbarriere), startet Welle 2: Sie findet die User über das Aktenzeichen wieder, ändert die Miete (optimistisches Locking, Version 1 → 2) und ordnet die Auszahlung mit dem Betrag der *neuen* Miete an. Das GUI-Feld **Aufträge** ist hier die User-Zahl — je User laufen zwei Tests, das Standard-Preset 20 Mitarbeiter / 2000 User ergibt also 4000 Transaktionsblöcke. Referenzlauf: 4000/4000 verifiziert in 13 Sekunden; der Datenbank-Endzustand wurde per SQL bestätigt (2000 Vorgänge, alle Fälle exakt Version 2, 2000 Zahlungen ANGELEGT, Summe deckungsgleich mit dem Gesamtreport).

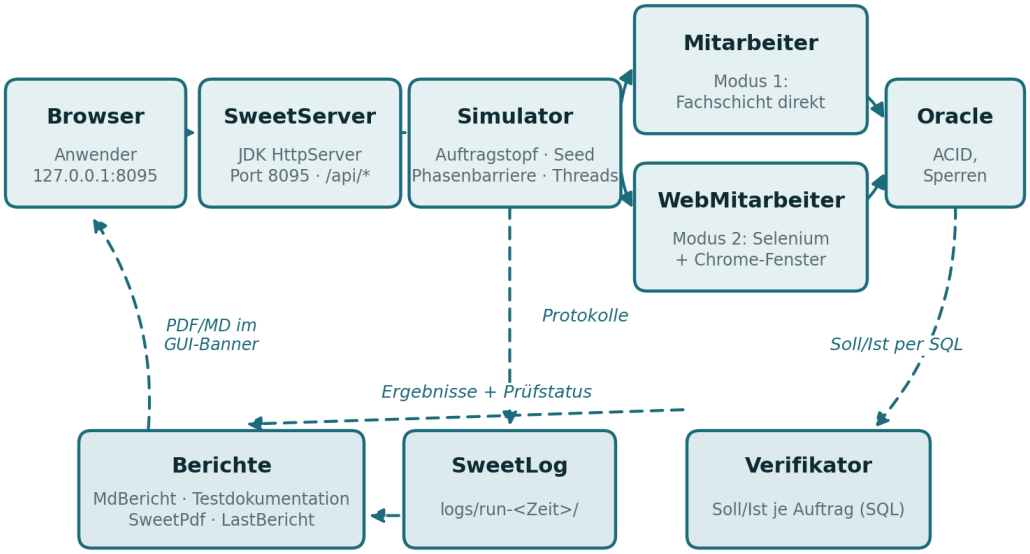
7 Regeln für den sicheren Umgang

- **Nur lokal betreiben:** Die Oberfläche lauscht ausschließlich auf 127.0.0.1, hat keine Anmeldung und erlaubt Löschungen (Aufräumen). Sie ist ein Entwicklungswerkzeug — niemals fernzugänglich betreiben.
- **Datenhygiene:** Alle Vorgänge tragen das Aktenzeichen-Präfix `SWEET-` (je Kategorie mit Zusatz, siehe Tabelle 2). Das Aufräumen vor dem Lauf entfernt nur diese Reste — komplett, mit Kindtabellen zuerst. Der Seed-Bestand der Fachdatenbank bleibt unberührt.
- **Nur gegen die Test-Datenbank:** Die Szenarien buchen echte Zahlungen, Storni und Historisierungen im angeschlossenen Schema. Die Entwickler-Docker-Oracle ist der richtige Ort; gegen Bestands- oder Produktivdaten gehört der Simulator nicht.
- **Globale Prüfungen zählen:** Zu jedem Lauf gehören globale Datenbankprüfungen. Eine Abweichung oder ein SQL-Fehler dort setzt den Lauf auf FEHLGESCHLAGEN — auch wenn einzelne Aufträge grün waren — und verhindert ein erfolgreiches Ergebnis in GUI, PDF und Markdown.
- **Modus 2 und Chrome:** Jedes Selenium-Fenster bekommt ein zufälliges, registriertes Chrome-Profil unter `testsweet/logs/browser-profile/`. Das Aufräumskript beendet nur

Chrome mit exakt passendem registrierten Profilpfad; fremde Browserfenster werden nicht angetastet. Nach einem Sitzungstod werden bestätigte Arbeitsschritte nicht blind wiederholt — bei unklarem Commit-Ausgang eines Schreibschritts schlägt der Auftrag mit Hinweis fehl.

8 Architekturüberblick

Technisch ist testsweet ein kleines, bewusst framework-freies Java-Programm (22 Klassen, rund 7.800 Zeilen im Hauptteil, dazu fünf Testklassen), das zwei Rollen hat: Es ist **Webserver** für die Dashboard-Oberfläche und **Orchestrierung** für die simulierten Mitarbeiter. Das folgende Bild zeigt die Komponenten und ihre Leitungen:



Alle Komponenten im Paket de.allgemein.testsweet — 22 Klassen, rund 7.800 Zeilen Hauptcode, dazu 5 Testklassen.

Abbildung 6: Klassen und Zusammenspiel — vom Browser über SweetServer und Simulator zu den Workern, darunter Protokoll, Verifikation und Berichte

So liest man das Bild: Der solide Fluss oben ist der Weg eines Laufs: Der Browser spricht die Endpunkte `/api/start`, `/api/status`, `/api/testfaelle`, `/api/auftraege`, `/api/pdf`, `/api/md` und `/api/stop` an — alle auf dem JDK-HttpServer von `SweetServer`. Der `Simulator` baut aus der `SzenarioBibliothek` den Auftragstopf (mit Seed reproduzierbar) und füttert damit N Worker-Threads: `Mitarbeiter` im Modus 1 direkt auf die Fachschicht, `WebMitarbeiter` im Modus 2 über Selenium in eigene Chrome-Fenster. Beide landen zwangsläufig in derselben Oracle-Datenbank — genau dort soll geprüft werden.

Der gestrichelte Fluss unten ist der Nachweis: `SweetLog` schreibt je Lauf ein Verzeichnis mit Protokollen; `Verifikator` und `GlobaleVerifikation` prüfen Soll gegen Ist per SQL; `MdBericht`, `Testdokumentation`, `SweetPdf` und `LastBericht` erzeugen daraus Markdown und PDF — genau die Dateien, die das GUI-Banner anbietet.

9 Der Technologiestack

testsweet folgt derselben Philosophie wie das Hauptprojekt: bewusst keine Frameworks, alles nachlesbar. Die Abhängigkeiten in `testsweet/pom.xml` sind überschaubar:

Tabelle 3: Bausteine und ihre Rolle

Baustein	Version	Rolle im Simulator
Java	21	Laufzeit und Sprache; Thread-Parallelität über die Standardbibliothek
transaktionssteuerung	0.5.8	das getestete Hauptprojekt: Fachschicht, TxManager, DAOs — Modus 1 ruft es direkt auf
Oracle JDBC (ojdbc11)	via Hauptprojekt	Datenbankzugang für die Szenarien und die SQL-Verifikation
selenium-java	4.27.0	Modus 2: Steuerung der sichtbaren Chrome-Fenster (Anmeldung, Masken, Freigabe, Storno)
WebDriverManager	5.9.0	beschafft den passenden ChromeDriver automatisch und hält einen Ersatztreiber bereit
JDK HttpServer	JDK-Bestandteil	die Dashboard-Oberfläche auf Port 8095 — ohne Web-Framework
JUnit Jupiter	5.10.2	nur für die eigenen Tests des Simulators (siehe Kapitel 12)
SweetPdf (Eigenbau)	—	PDF-1.4-Erzeugung ohne Drittbibliothek; bindet Grafiken als native XObjects (DCTDecode)
Python + matplotlib	Werkzeugkette	erzeugt die Fachflussdiagramme (testsweet/generate_diagrams.py, 200 dpi)

Zwei Details stechen heraus. Erstens: Der Simulator testet *Produktionscode mit Produktionscode* — Modus 1 hängt an denselben Klassen wie die Fachanwendung, nicht an einer Kopie. Zweitens ist selbst der PDF-Export eine Eigenentwicklung (`SweetPdf`), damit der Bericht ohne schwergewichtige Bibliotheken aus einem einzigen Prozess fällt. Die Chrome-Fenster des Modus 2 laufen mit verkleinerter Darstellung (`--force-device-scale-factor=0.8`), damit die Seiten auch im kleinen Rasterfenster vollständig sichtbar sind.

10 Die Klassen

Die 22 Hauptklassen im Paket `de.allgemein.testsweet` verteilen die Verantwortung auf klare Rollen — von der Szenario-Definition über die Ausführung bis zu Protokoll und Bericht:

Tabelle 4: Alle Hauptklassen (Zeilenstand 12.10.2026)

Klasse	Zeilen	Rolle
SzenarioBibliothek	1752	die Szenario-Arten des Modus 1 samt Erwartungshaltung (Beträge, Status, Versionen)
Simulator	934	Orchestrierung: Auftragstopf, Worker-Threads, Phasenbarriere (Stress), globale Prüfungen
SweetPdf	738	eigenentwickelter PDF-Generator (PDF 1.4) für Testdokumentation und Lastreport
Testdokumentation	592	baut den Positiv-/Negativ-Bericht: je Fall Rechenweg, Erwartung/Ist-SQL, Ablauf
WebSzenarioBibliothek	577	die Web-Szenario-Arten des Modus 2 (inkl. WEB-NEG-Varianten)
WebHelfer	541	Selenium-Werkzeuge: Klicks, Wartezeiten, Screenshots, Wiederaufnahme nach Sitzungstod
SweetServer	401	HTTP-Oberfläche auf Port 8095 (JDK HttpServer) samt /api-Endpunkten
LastBericht	367	der gekürzte Aggregate-Report für Last- und Stresstest
SweetLog	300	Lauf-Protokollierung: run-Verzeichnis, je Mitarbeiter/Auftrag, Verifikations- und Fehlerlog
TestfallKatalog	294	Katalog der Testfallbeschreibungen für den Dialog „Testfälle im Detail“
WebMitarbeiter	268	ein simulierter Sachbearbeiter im Modus 2 — fährt sein eigenes Chrome-Fenster
MdBericht	262	erzeugt die maschinenlesbare Markdown-Fassung der Berichte
Mitarbeiter	138	ein simulierter Sachbearbeiter im Modus 1 — ruft die Fachschicht direkt auf
Auftrag	116	Wertobjekt: Szenario, Aktenzeichen, Hauptperson, Erwartung und Status
SweetMain	115	Einstiegspunkt: CLI-Auswertung (run/web) oder Start der GUI
Verifikator	110	Soll/Ist-Prüfung je Auftrag per SQL (Beträge, Status, Budget, Versionen)
BrowserProzesse	109	Chrome-Prozess- und Profil-Verwaltung samt sicherem Aufräumen

Klasse	Zeilen	Rolle
GlobaleVerifikation	49	globale Datenbankprüfungen zum Laufresultat (Abweichung → FEHLGESCHLAGEN)
Erwartung	48	Wertobjekt: der erwartete Datenbank-Endzustand eines Auftrags
Rechenblock	42	Wertobjekt: Zwischenwerte der Bedarfsberechnung für Dokumentation und Doppelklick-Ansicht
SzenarioAktion	38	eine Aktion eines Fachschicht-Szenarios (Schritt im Auftrag)
WebAktion	32	eine Aktion eines Web-Szenarios (UI-Schritt im Chrome-Fenster)

10.1 Die wichtigsten Klassen im Detail

`SzenarioBibliothek` und `WebSzenarioBibliothek` sind das fachliche Herz: Jedes Szenario trägt seine Erwartungshaltung in sich — die Beträge, Status und Versionen, die am Ende in der Datenbank stehen müssen. Die Erwartung wird vorab mit derselben `Leistungsberechnung` gerechnet, die auch die Fachschicht verwendet; deshalb kann ein Verifikationsfehler nur zwei Ursachen haben: rechnet die Fachschicht falsch, oder wurde nicht das gebucht, was gerechnet wurde.

`Simulator` ist der Dirigent: Er mischt den Auftragstopf (Seed!), startet die Worker-Threads, setzt beim Stresstest die Phasenbarriere zwischen Anlegen und Bearbeiten, lässt den Kassenlauf bewusst parallel zu den Freigaben laufen, sammelt die Ergebnisse und fährt die globale Verifikation. `Mitarbeiter` und `WebMitarbeiter` sind die Arbeiter — einer kürzer, einer mit Browser: Beide ziehen sich Aufträge aus dem Pool, spielen sie Schritt für Schritt (`SzenarioAktion` bzw. `WebAktion`) und schreiben ihre Spuren über `SweetLog`.

`Verifikator` und `GlobaleVerifikation` sind das Rückgrat des Nachweises: Ersterer prüft je Auftrag den erwarteten Endzustand per SQL (auch negativ: nichts darf entstanden sein), Letztere zusätzlich den Gesamtbestand des Laufs. Ihre Ergebnisse fließen in GUI, `zusammenfassung.txt` und beide Berichtsketten. Die Berichtskette selbst ist dreigeteilt: `Testdokumentation` baut den Inhalt (Rechenweg, Erwartung/Ist-SQL je Fall), `MdBericht` schreibt ihn als Markdown und `SweetPdf` als PDF; `LastBericht` erzeugt die schlanke Aggregate-Fassung für Last und Stress.

`WebHelfer` und `BrowserProzesse` kapseln die Sprödigkeit echter Browser: stabile Klick-Hilfen (Element sichtbar machen, per JavaScript klicken), Screenshots je Schritt, Treiber-Neustart mit Ersatztreiber, und das saubere Aufräumen, das ausschließlich Chrome-Prozesse mit registriertem Profilpfad beendet. `SweetServer` schließlich ist der dünne Web-Faden, der alles zusammenbringt — CRUD für Status und Auftragsliste, Streaming des Live-Protokolls und die Download-Endpunkte für die Berichte.

11 Abläufe im Detail

Die folgenden Fachflussdiagramme erzeugt `testsweet/generate_diagrams.py` automatisch (200 dpi); dieselben Grafiken stecken auch in der Testdokumentation und in der GUI-Galerie.

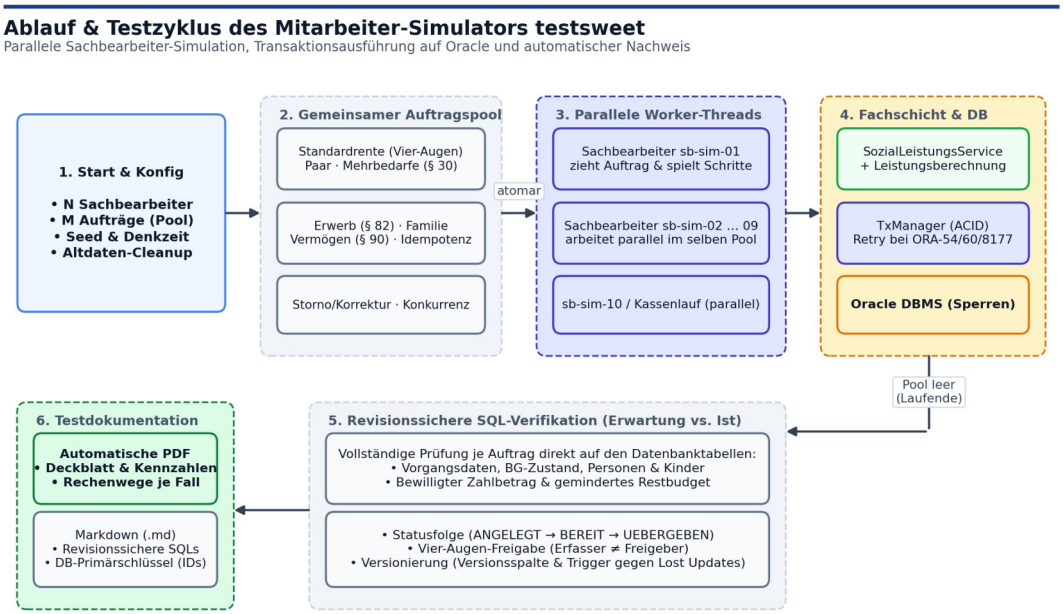


Abbildung 7: Ablauf und Testzyklus des Mitarbeiter-Simulators

So liest man das Bild: Von links nach rechts: Start und Konfiguration (N Sachbearbeiter, M Aufträge im Pool, Seed und Denkzeit, Altdaten-Aufräumen), der gemeinsame Auftragstopf mit den Szenario-Arten, die parallelen Worker-Threads, und rechts das Ziel — Fachschicht und Datenbank. Der Rücklauf unten schließt den Zyklus: Verifikation, Protokolle, Berichte.

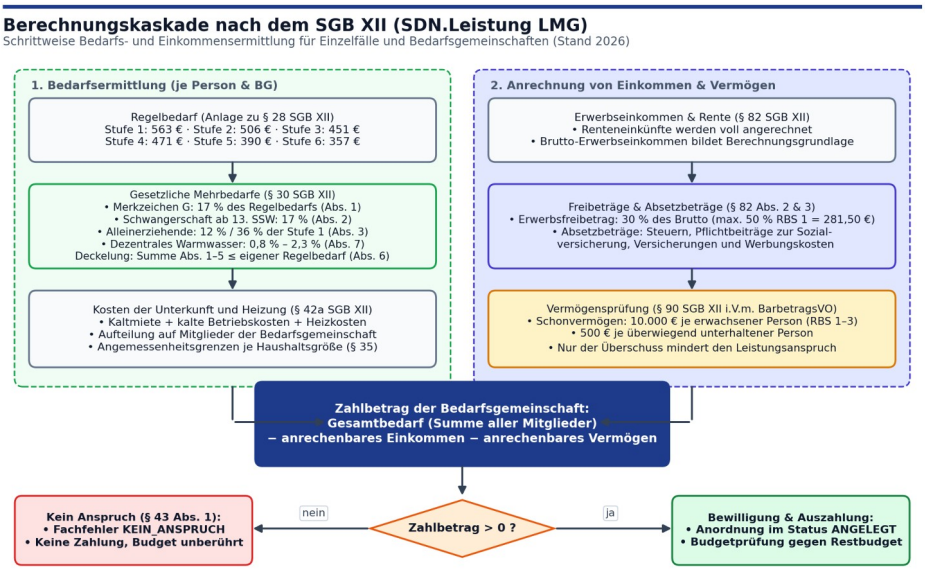


Abbildung 8: Die Berechnungskaskade nach SGB XII — die Grundlage jeder Szenario-Erwartung

Dieses Bild erklärt, woher die erwarteten Beträge der Szenarien kommen: dieselbe Kaskade von der Regelbedarfsstufe über Mehrbedarfe, Kosten der Unterkunft, Absetzbeträge und Vermögen bis zum Zahlbetrag. Weil Simulator und Fachschicht dieselbe `Leistungsberechnung` nutzen, ist die Erwartung keine Sekundär-Implementierung, sondern ein Gegencheck derselben Rechtslogik.

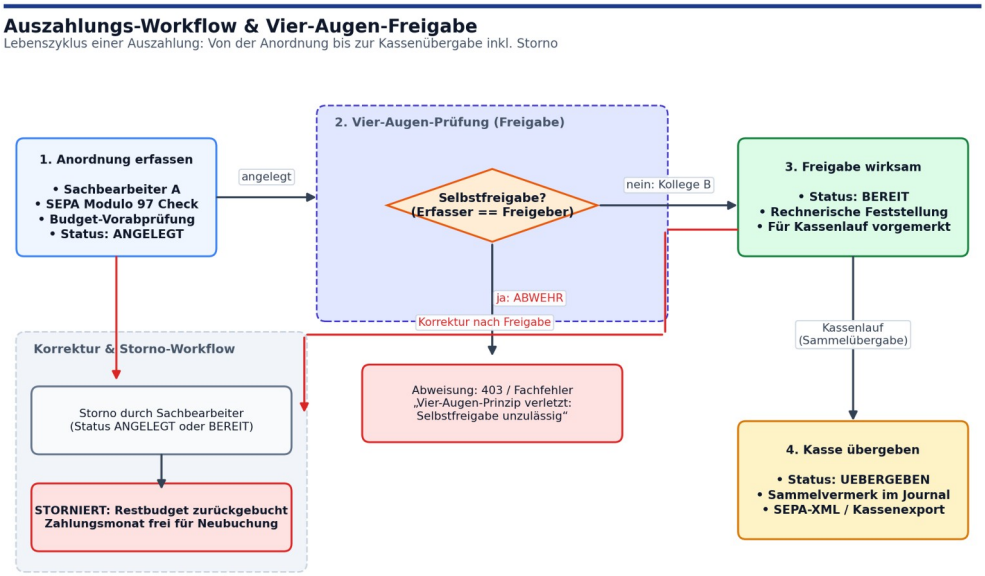


Abbildung 9: Auszahlungs-Workflow mit Vier-Augen-Freigabe und Storno

Der Vier-Augen-Workflow ist der meistgeprüfte Weg im Simulator: Anordnen (ANGELEGT), Freigabe durch eine zweite Person (BEREIT), Übergabe im Kassenlauf (UEBERGEBEN) — und als Abzweig das Storno mit Restbudget- und Vermögenseinsatz-Rückbuchung. Genau diese Kette spielen die Positiv-Szenarien Standardrente und die Web-Varianten durch; die Negativ-Varianten prüfen die Abwehrseiten (Selbstfreigabe, Doppel-Freigabe, Storno nach Kasse).

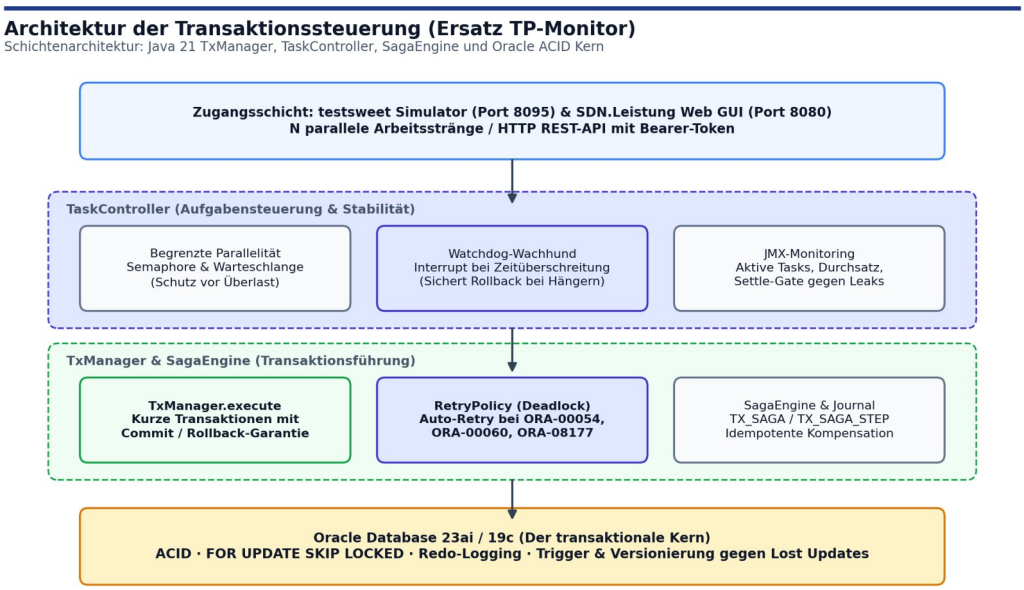


Abbildung 10: Transaktionssteuerung und Oracle-ACID-Kern

Der Stresstest richtet sich gezielt gegen diese Schicht: 4000 Transaktionsblöcke mit Sperren auf dieselben Vorgänge, Optimistic-Locking-Wechsel (Version 1 → 2) und Budgetführung unter Konkurrenz. Die Retry-Logik bei Sperrenkonflikten (ORA-00054/60/8177) läuft dabei im getesteten `TxManager` — der Simulator überspringt sie nicht.

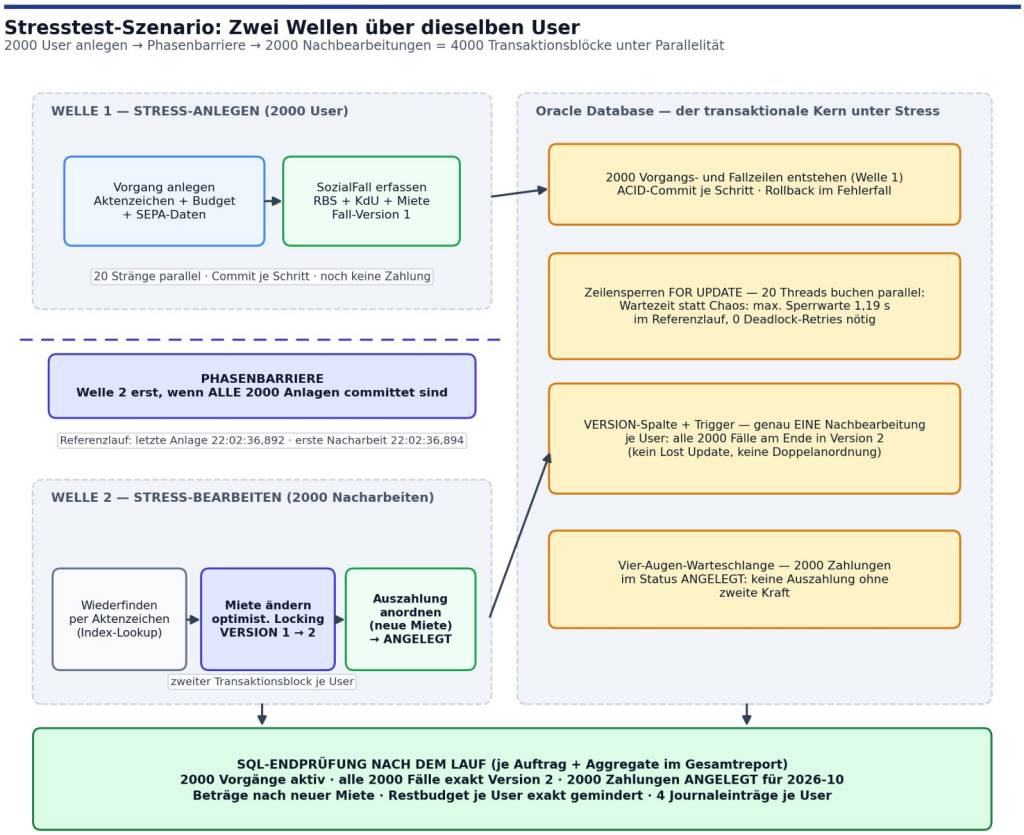


Abbildung 11: Das Zwei-Wellen-Schema des Stresstests mit Phasenbarriere

So liest man das Bild: Welle 1 legt die Bestände an (Vorgang + SozialFall, noch ohne Zahlung), die Phasenbarriere sorgt dafür, dass Welle 2 erst startet, wenn Welle 1 vollständig durch ist — sonst wäre die Änderung der Miete in Welle 2 ein Wettrennen mit der Anlage. Welle 2 beweist dann den Kern des Lost-Update-Schutzes:lesen unter Sperrung, Änderung mit Versionsprüfung, Anordnung nach neuer Miete.

12 Die eigenen Tests des Simulators und bekannte Grenzen

Der Simulator testet zwar andere, hat aber auch fünf eigene Testklassen (`src/test/java/de/allgemein/testsweet/`), die über `mvn -f testsweet/pom.xml test` laufen: `GlobaleVerifikationTest` (globale Prüfungen setzen den Lauf korrekt auf FEHLGESCHLAGEN), `WebWiederaufnahmeTest` (Sitzungstod im Modus 2: bestätigte Schritte werden nicht wiederholt), `TestfallKatalogTest` (der Katalog der Testfallbeschreibungen ist vollständig und passt zu den Szenarien), `MdBerichtFilterTest` (die Markdown-Erzeugung filtert korrekt je Kategorie) und `BrowserProzesseTest` (das Aufräumen berührt nur registrierte Chrome-Profile).

Bekannte Grenzen, bewusst dokumentiert: Die Storno-Spurenprüfung ist keine vollständige Rollback-Bilanz eines mehrstufigen Auftrags. Die GUI-Kenndaten zeigen nur den letzten Lauf der eigenen Sitzung. Der Stresstest läuft bewusst nur im Fachschicht-Modus — ein Selenium-Stresstest würde vor allem den Browser-Stack testen, nicht die Fachlogik. Und die Berichte für Positiv/Negativ dokumentieren jeden Fall einzeln, weshalb Last und Stress bewusst nur Aggregate schreiben.

Damit ist der Bogen geschlossen: testsweet fährt die Anwendung so, wie sie später benutzt wird — mit echten Abläufen, echten Sperren und echten Browsern — und liefert für jede Behauptung einen nachprüfbaren Beweis: als SQL, als Protokollzeile, als Screenshot oder als Seite im PDF-Bericht.